

Project Title: Internship Placement Management System

Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course (SC2002/CE2002 CZ2002)	Lab Group	Signature /Date
Justin Hotasi Napitupulu	SC2002	SCE4	 17/11/2025
Cheah Jia Tai			 17/11/2025
Lim Seow Kiat			 17/11/2025
Caasi John Rian Domingo			 17/11/2025
Bian Lingzhu			 17/11/2025

Chapter 1: Requirement Analysis & Feature Selection

We began by analyzing the Internship Placement Management System (IPMS) assignment brief and identified three main user roles, namely, Student, Company Representative, and Career Staff, each having unique permissions and responsibilities. The objective of this IPMS is to stimulate NTU's internship placement workflow using a CLI-based, object-oriented Java program.

1.1 Explicit Requirements

All users have an ID, name, and password and can perform basic actions such as login and changing passwords. Each role however, supports different capabilities:

Students can view only approved and visible internships that match their major and year level. Lower year students are restricted to basic level internships, while senior students can access more advanced roles. Students may take up to three internships, accept one approved offer and submit withdrawal requests. Once a student accepts an offer, the system

automatically withdraws all remaining active applications

Company Representatives may register for an account (subject to Career staff approval), create up to five internship postings, and edit or delete their own postings as long as these have not yet been approved. They can also view all applications submitted to their internships and see associated student details.

Career Staff act as administrator. They approve or reject new company representatives, validate internship postings, and process student withdrawal requests. They have full visibility across all users, internships and applications

1.2 Implicit Requirements

Beyond the explicit task described, the system's behavior implies several additional requirements for a realistic and consistent workflow. The following implicit requirements are:

Rule-Based Access Control: Students should not be able to modify the internships or view other users' data. Company reps can only manage internships under their company and see applications tied to those internships. Career staff have full oversight and approval authority across all entities.

Application Lifecycle and Consistency: The use of application statuses in our model implies: A student application normally starts as PENDING. If a company approves it, it becomes SUCCESSFUL. If it is rejected, it becomes UNSUCCESSFUL. If the student no longer wishes to continue, they could withdraw and ultimately result in WITHDRAWN when processed. When a student formally accepts an offer, the application moves to ACCEPTED

Internship Visibility and Eligibility: Internships are not visible to students until approved and explicitly marked as visible. Students only see internships that match their major and when relevant, their year of study. Year 1-2 students should be restricted to basic internship levels, while higher year students may access more advanced postings.

1.3 Interpreting and Resolving Ambiguous Parts (Assumptions)

Company Rep Rejection: To allow for the rejected Reps to know their status, it will remain in the datastore with status set to "REJECTED". This is shown upon login and can be seen in

the authcontroller.

Max 5 Internships for Company Rep: We assumed that those 5 included pending or approved internships. Rejected or filled internships do not contribute to this limit.

Handling Multiple Offers for the Same Student: We decided that students are allowed to hold multiple successful applications, but once the student accepts one offer, the system automatically withdraws other pending or successful applications.

1.4 Core Features based on requirements

User Authentication and Role Separation: Centralized login flow for all the 3 users (students, company representatives, career staff). Session tracking for the currently logged in user. Clear separation of menus and actions depending on the user's role.

Internship Management: Representation of internships with key attributes such as title, description, preferred major, company name, open/close date, status, visibility, and capacity. A controlled lifecycle for internships which includes pending approval, rejected and filled states. A slot based mechanism that automatically marks an internships as filled when all slots are taken

Student Application Workflow: Students can view internships that they are eligible for based on approval status, visibility, major and level. They may submit applications, view applications by status, accept successful offers and request withdrawals. Accepting an offer updates the internship's filled slots and triggers auto withdrawal of the student's remaining applications.

Data Loading and Consistency: All users, internships, and applications are initialized from CSV files at program start. Each CSV row is then converted into a corresponding object, which is added into a dedicated store (user store, internship store, etc.) Internships and applications are relinked to user accounts using their IDs to ensure consistency.

Role Based Viewing of Internships and Applications using Filters: Instead of hardcoding specific queries, our system uses flexible filtering criteria for internships and applications. Students see only valid internships, Company representatives see only their own internships,

and Career staff should be able to see all internships and applications.

1.5 Additional features in our implementation

InputHandler: We introduced a dedicated InputHandler class to centralize all user input validation. This class uses static methods and try except blocks to ensure robust input parsing and consistent error handling across all of our UIs.

Internship Slot management: Our system manages internship slot allocation automatically. Each internship tracks filledSlots and updates its status to FILLED once capacity is reached. This helps prevent over allocation and maintain data integrity when students accept offers.

IDataStore Abstraction: We introduce IDataStore Abstraction to promote loose coupling and better testability. This abstraction makes the data layer more modular and allows future extensions or alternative storage implementations without modifying existing controllers.

Change Password Re-login Flow: When users change their password, the system immediately logs them out and requires re-authentication. This enhances security and ensures that the new credentials take effect cleanly.

Chapter 2: Our Implementation

Our system was structured using a 3-layered architecture that strongly implemented Boundary, Entity, and Controller classes. It closely resembles the model-view-controller pattern (MVC), where we separate the application's concerns into a logic class (controller), the entities, and the view (UI), where user interfaces should only consist of logic pertaining to display, and not contain any business logic. In addition, we grouped similar classes in function within packages (UI classes under boundary, etc) to further enforce encapsulation and cohesion. Below is the list of packages/class types we implemented:

User Classes: Contains User entity classes such as Student

Controller Class: Contains classes responsible for handling logic. This logic will then be passed on to boundary classes for display.

Boundary classes (UI): Consists of our user interfaces. Uses processed data from controllers to display.

Common Boundary Classes: Common views are implemented as static so as to be reusable among user UIs. Internship and application display can be common amongst UI.

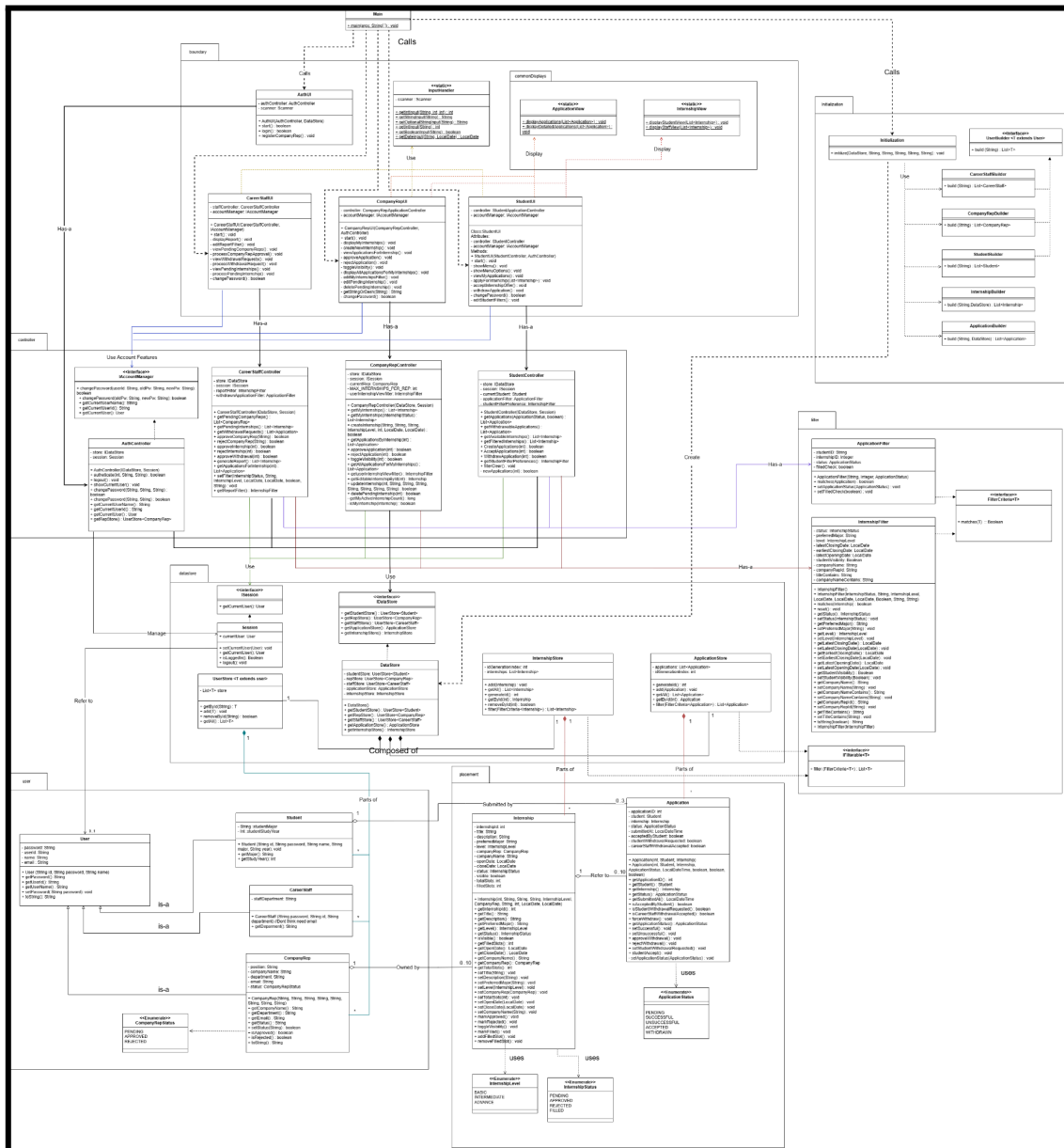
Placement classes: Entities related to the internship workflow. The object classes.

Filter classes: Consists of classes that help with filtering our various placement classes

Datastore classes: Consists of classes that help with storing our user and placement classes

Initiation classes: Consists of classes that help with extracting information in csv files, transforming them into classes

2.1 Our UML class diagram



Our UML class diagram is created with the following focus:

- Readability by ensuring top-down flow and naming based on the class's responsibility
- High Cohesion by ensuring each class has a clear responsibility

- Low coupling is achieved by ensuring all classes only have necessary relationships with other classes.

2.1.1 How were relationships (inheritance, association, aggregation) determined?

Inheritance (is-a) (User-Each Roles)

Since these classes share similar attributes (userID, password) and base functions. It was logical to group these into an inheritance relationship. This way, we can upcast it into a user whilst maintaining common user-related functions. This results in code reusability. Any other specialized function can simply be implemented in its respective class.

Association (uses-a) (Controller-Filter)

Class-specific controllers use a filter object to determine the correct filter logic to generate and display to the UIs. For example, controllers such as CompanyRepController, StudentApplicationController, and CareerStaffController would need to filter and display internships. Therefore, it uses the corresponding filter.

Aggregation (has-a) (Internship-Application & CompanyRep)

An application will have a reference for a student and an internship. These parts belong to an application, but they can also exist independently. Hence, if an application itself is deleted, the student and internships would not be deleted. This is also why it is determined that it is not a composition and instead an aggregation.

Composition (stronger has-a) Application (DataStore-Individual Stores)

The datastore and other stores is an example of a strong has-a relationship, which we used to centralize all our individual data stores into. Each store is an integral part of DataStore and cannot exist independently. Without DataStore, each store will have no place to exist. Similarly, all applications must be inside Application Stores, and will perish without the stores due to redundancy. This design ensures centralized management and dependent lifecycles for core data components.

2.2 UML Sequence Diagram

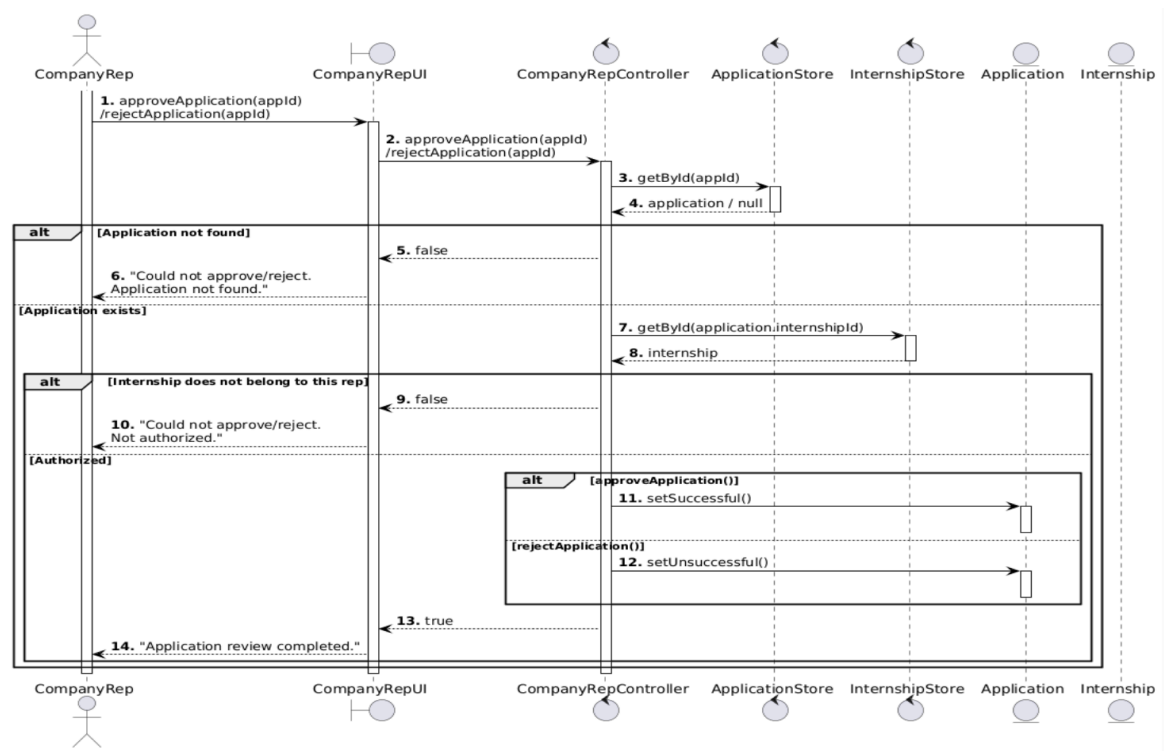
To complement our class diagram and illustrate the dynamic behaviour of the system, we developed sequence diagrams that capture the key interactions between users, boundary classes, controllers, and data stores. These diagrams highlight how different system

components collaborate to fulfil major use cases, such as internship applications and company representative approvals.

2.2.1 Sequence Diagram for “Company Representative Reviews an Application”

1. The company representative will choose to either approve or reject an application through the UI.
2. The UI then sends a request to the controller, which will retrieve the application by using the application ID.
 - If the application does not exist, the system will return an error to the representative.
 - If the application exists, the controller will retrieve the associated internship.
3. The controller verifies whether the internship belongs to the representative’s company.
 - If it is unauthorized, the system will reject the action.
 - If it is authorized, the controller applies the representative’s decision by updating the application status (either successful or unsuccessful).
4. The system confirms completion of the review to the representative.

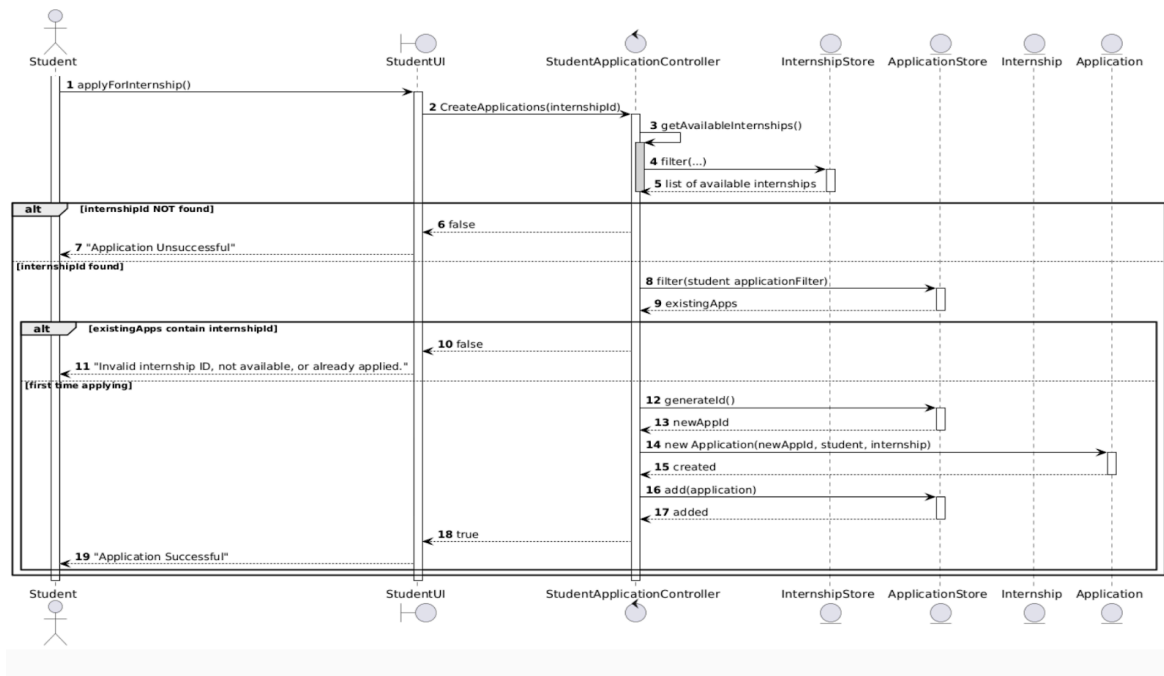
This completes the sequence of “company representative reviewing an application”.



2.2.2 Additional Sequence Diagram for “Student applies for internship”

1. The student initiates an internship application through the Student UI.
2. The UI forwards the request to the controller which will then retrieve the list of available internships.
3. The controller verifies whether the selected internship is available.
 - If valid, the controller will load and list all applications previously submitted by the student.
 - If invalid, the system will return an unsuccessful application response to the student.
4. The controller then checks whether the student has already applied for the selected internship.
 - If already applied, the system returns an error message.
 - If it is their first time applying for that internship, then the controller will generate a new application ID. It will also create the Application object, and store it.
5. The system will return a successful application confirmation to the student.

This completes the sequence of “student application submission”.



Chapter 3: Object-Oriented Design Considerations

3.1 Application of OODP concepts

Extensibility: Our system makes use of the User parent class as inheritance for future extensibility. In the case where another user type is required, our system can extend the functionality of the base user and in its own class, implement specific functions.

Maintainability: Separation of concerns is our main focus in upholding maintainability within our code base. By separating the functions into their specific roles (controllers / UI). We isolate the issue to a single layer, making it easy to identify bugs quickly. This also means that our classes have low coupling. In addition, if we needed to change any business rule pertaining to how the company rep handles internships. We will intuitively know that it is within the CompanyRepController as we know it handles all the logic involving the company rep. So in the case that we need to set the max internships that the Rep can set, we would only need to change the controller class.

Single Responsibility Principle (SRP): We began by structuring the system so that each class focuses on one clearly defined responsibility. This is visible in several components, for example, classes were divided into entity, controller and boundary classes, to fulfill their responsibilities of handling entity data, handling the core logic that runs our project, and handling interactions between the system and external environment. We even went on to create classes like filter and datastore classes to further split important responsibilities like filtering and containing + retrieving our entity classes, ensuring easy maintainability of the code and low coupling between classes.

Open/Closed Principle (OCP): One example of this principle is that we created a User class, which CompanyRep, Student and CareerStaff extends from. This way when we want to add a new functionality, like creating a user-type class that behaves differently from our existing classes, we will not need to modify our existing user classes and instead create a new class by extending from User class, ensuring our existing code is closed for modification but open for extension. This principle can also be seen in our UserStore class, where we used generic code to promote extensibility and prevent the need to modify our code.

Liskov Substitution Principle (LSP): We ensured that all subclasses can be replaced/substituted when their parent class is expected. They do so safely without breaking the program's behaviour. In our system, Student, CompanyRep, CareerStaff all extend User. They inherit its behaviour such as password validation. Our system remains consistent when substituting such that no functionality would be lost.

Interface Segregation Principle (ISP): By using the IAccountManager interface, our UI classes do not need to use the other unrelated logic in AuthController. Since the UIs will only need a subset of functions (changePassword(), getUsername()), the interface exposes a specific subset that is required for the UI to perform its actions (change password). And if another login method is required (external login), we can simply change the authcontroller that is passed in to implement IAccountManager. The same principle applies to the ISession interface, only providing the necessary method (getCurrentUser) for roles-controllers.

Dependency Inversion Principle (DIP): We ensured that high-level components depend on abstractions rather than concrete implementations. For example, our boundary (higher level) classes depend on controller classes through their abstract behavior rather than their internal implementation details. Our controller classes also depend on abstractions such as IAccountManager, IDataStore, ISession, and filter interfaces. This structure means that our high-level components do not instantiate or tightly couple itself to specific underlying classes.

Encapsulation: Data and attributes within each of the classes are only accessible through the getters and setters of the class. Where all attributes have been declared as private and can only be accessed through the respective functions. In addition, functions that do not need to be exposed due to business logic are set to private. For example, setApplicationStatus should not be accessed publicly, instead they should use the public functions setSuccessful, setUnsuccessful etc to adhere to business logic using controlled setters.

Static Utility Classes: We have identified certain common features within the user interfaces regardless of their role. For example, displaying internships and applications. There is a static boundary class named ApplicationView which contains static functions that display application details. This display is common among all users, in that they are to be displayed the same way in showing application ID, student, company rep etc. In addition, we have

implemented our own input handler class which manages our system's scanner input and throws the necessary exceptions and input validation so we do not have to reuse them everywhere.

Polymorphism: We have used polymorphism in the form of interface throughout the implementation. The strongest case of polymorphism is the User super class, which allows classes that utilize User, like ISession and IAccountManager, to be used for all three controllers without issue.

3.2 Design Trade-offs

Database Access: We chose to avoid storing calculated attributes inside the entities. This included data such as currentApplication count or holding the actual object itself. For example, with students, we did not choose to contain a list of sentApplications into the student object. Instead we always query the datastore whenever information is required. This way, we will guarantee that data retrieved by the datasource is always consistent as it is the single source of truth for data. Classes that depend on the "application" class for example will check and filter based on status there, rather than accessing from a student object which may potentially be off-sync when there is human-error when coding in the conditions for storing pending applications. The downside of this approach is that it performs frequent lookups in the datastore which results in performance overhead. However we deem these as critical data and therefore would wish to enforce data integrity above optimisation.

Centralised Creation of Controllers: We chose to centralise the creation of the main controllers of the application. All controllers will be instantiated once during our system startup, and subsequently, any controller that depends on another explicitly receives it as a parameter in its constructor. Although it will require passing in more variables when creating controllers, it makes controller dependencies implicit, making it easy for developers to see what dependencies a controller actually needs. This is in contrast with the idea of implementing a Singleton pattern, whereby we create controllers into a singleton and expose them as a global static method. This would introduce tighter / hidden coupling, and tracing bugs more difficult as we would have to look into the code in each controller to see whether a static method has been used or not.

3.3 Contribution of Design Patterns to Overall System Design

By applying the SOLID principles, it resulted in being more modular and maintainable, resulting in easier coding and testing overall. This is especially made so by the Single Responsibility Principle during development. If an entire entity class is yet to be implemented, the UI and other unrelated sections will continue to work independently. This allows us to implement each class's core responsibilities without needing to wait for another. This also shows how loosely coupled and cohesive our classes are designed. In summary, applying SOLID principles and consciously thinking about design choices help create a maintainable, extensible and easy to understand codebase.

Chapter 4: Test cases

We created mock csv for interviews, applications and CompanyRep for faster test cases. We tested each test case manually, showcasing all features are functional as stated within one run of the system. In summary, all the test cases mentioned in Assignment's appendix A are successful for our code, see more in detail (Group 4 test cases.pdf) inside our Git repo (Appendix A).

Chapter 5: Reflections throughout the Assignment

5.1 Difficulties Encountered and Reflection

Managing practical time constraints was one of the more challenging difficulties. Developing features concurrently, though possible, still required on other features in testing. And with conflicting time tables, it made the project harder to complete. The importance of the class diagram is also understated, in that it is very difficult to make changes once the overall structure has been created. This in turn affected our coding methodology especially when implementing the SRP or the OCP. Where we had to balance the trade off of clean separation of concerns or the added complexity of managing many small classes. Furthermore, a lot of foresight is required in order to design a solution that fully adheres to being "open" to extensibility.

5.2 Learning points + Suggestions for improvement

Stricter adherence to OCP and DIP by implementing more Interfaces / Abstract classes:

As we added more implementations and features, some of the additional features we had thought to implement late into the project, such as changing passwords, was something common between our (boundary) classes. We had not thought to implement interfaces or abstract superclasses for these classes due to the difference in behaviour in the basic

requirements, but late into the project, we definitely regretted not having implemented them because of these situations. Due to time constraints, we decided to not make such a drastic change late into the project, but for future projects, we would definitely implement more interfaces to build a solid foundation for the project, allowing for extensibility of our code.

Filter Implementation: In our current implementation, the filter class contains multiple attributes that perform all the filtering checks. This is functionally correct, but also reduces its extensibility as it handles checking too many filter conditions. The better implementation is to separate the filter criteria as specific and individual “strategy” (filter) class where we can make use of Java’s builder functionality to effectively create a flexible and readable way to combine filter conditions.

Appendix A: Link to our GitHub repository

<https://github.com/piemmoose/SC2002-SCE4-Grp4-Assg>